IPD441 - Visión por Computador

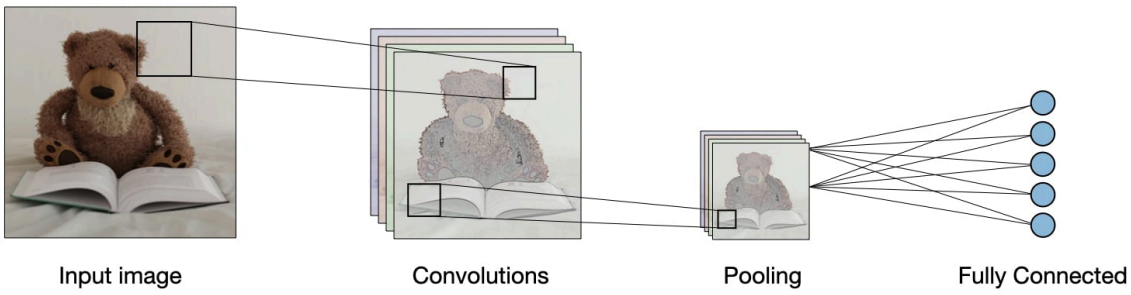
Clase 3: Recognition

Prof. Nicolás Torres
nicolas.torresr@usm.cl
Ingeniería Civil Telemática
Departamento de Electrónica
Universidad Técnica Federico Santa María

In [1]:
```
from Clase3 import *
```

Image classification

Es la tarea de asignar una etiqueta o categoría a una imagen. El objetivo es identificar qué hay en la imagen y clasificarla en una de varias categorías predefinidas. Por ejemplo, distinguir entre perros y gatos en imágenes de animales. Se utilizan arquitecturas de redes neuronales convolucionales (CNN).



VGG16, ResNet50, InceptionV3

VGG16 es más simple con 16 capas, mientras que ResNet50 tiene 50 capas y es conocido por introducir conexiones residuales para mejorar el rendimiento en redes profundas. InceptionV3 se encuentra en algún punto intermedio en términos de profundidad y complejidad.

In [2]:
```
model_vgg16 = VGG16(weights='imagenet')
model_resnet50 = ResNet50(weights='imagenet')
model_inceptionv3 = InceptionV3(weights='imagenet')
```

In [3]:
```
keras_models = {"vgg16": (224, 224), "resnet50": (224, 224), "inceptionv3": (299
```

In [4]:
```
recognition(keras_models, model_vgg16, model_resnet50, model_inceptionv3, "test/
```



Rank	VGG16	Score	ResNet50	Score	InceptionV3	Score
1	banana	35.59%	banana	72.18%	banana	94.99%
2	lemon	23.91%	orange	13.49%	orange	2.7%
3	orange	10.24%	lemon	10.87%	Granny_Smith	0.21%
4	pineapple	3.4%	Granny_Smith	0.89%	lemon	0.19%
5	bell_pepper	2.73%	pineapple	0.61%	pineapple	0.03%

In [5]:
```
recognition(keras_models, model_vgg16, model_resnet50, model_inceptionv3, "test/
```



Rank	VGG16	Score	ResNet50	Score	InceptionV3	Score
1	matchstick	17.04%	mountain_bike	26.92%	mountain_bike	48.4%
2	chime	6.58%	analog_clock	15.36%	unicycle	18.33%
3	flagpole	6.14%	wall_clock	13.66%	pole	2.07%
4	bell_cote	3.64%	bell_cote	5.01%	moped	2.05%
5	bow	2.98%	moped	4.14%	bicycle-built-for-two	1.83%

In [6]:
```
recognition(keras_models, model_vgg16, model_resnet50, model_inceptionv3, "test/
```



Rank	VGG16	Score	ResNet50	Score	InceptionV3	Score
1	dining_table	55.57%	dining_table	84.98%	folding_chair	97.51%
2	folding_chair	16.59%	folding_chair	7.86%	dining_table	1.4%
3	desk	14.57%	bookcase	2.47%	table_lamp	0.51%
4	wardrobe	2.14%	desk	2.35%	rocking_chair	0.2%
5	crib	1.21%	crib	0.19%	bookcase	0.02%

In [7]:
```
recognition(keras_models, model_vgg16, model_resnet50, model_inceptionv3, "test/
```



Rank	VGG16	Score	ResNet50	Score	InceptionV3	Score
1	Great_Dane	56.82%	whippet	20.85%	Weimaraner	8.61%
2	whippet	14.45%	Arabian_camel	10.27%	African_elephant	7.94%
3	Arabian_camel	11.63%	zebra	6.5%	tusker	6.94%
4	Italian_greyhound	6.45%	llama	5.54%	sorrel	3.77%
5	Saluki	2.26%	Weimaraner	5.01%	Great_Dane	3.72%

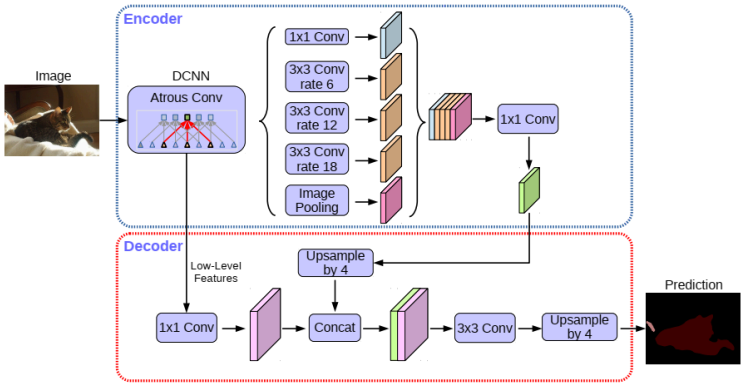
Semantic segmentation

Se centra en asignar una etiqueta a cada píxel de la imagen, lo que significa que se clasifica cada píxel en la imagen según la categoría a la que pertenece. El objetivo es entender la estructura y los contornos de los objetos en la imagen, lo que permite una comprensión detallada de la escena.

U-Net es conocida por su arquitectura de codificación-decodificación. FCN (Fully Convolutional Network) utiliza capas convolucionales completamente convolucionales. DeepLab emplea una estructura de atrás hacia adelante (decostructiva) para obtener segmentaciones precisas.

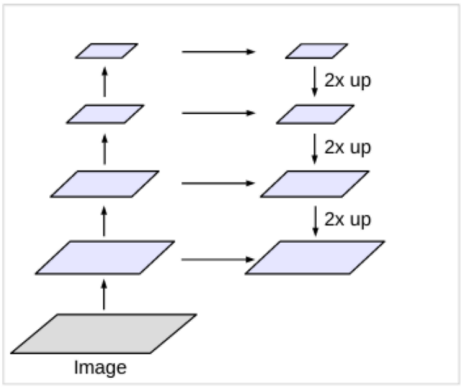
DeepLab

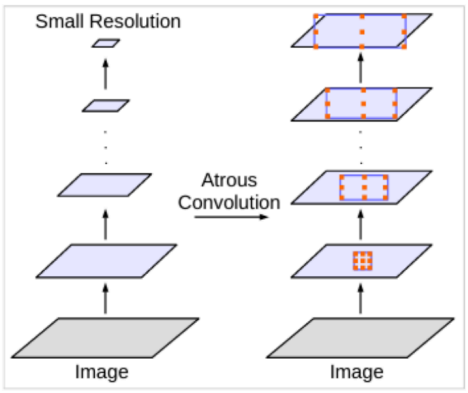
Utiliza convoluciones atrous para capturar información contextual a diferentes escalas y el bloque Atrous Spatial Pyramid Pooling (ASPP) para obtener representaciones ricas en contexto. Esto permite una segmentación detallada y precisa de las imágenes.



Arquitectura DeepLab

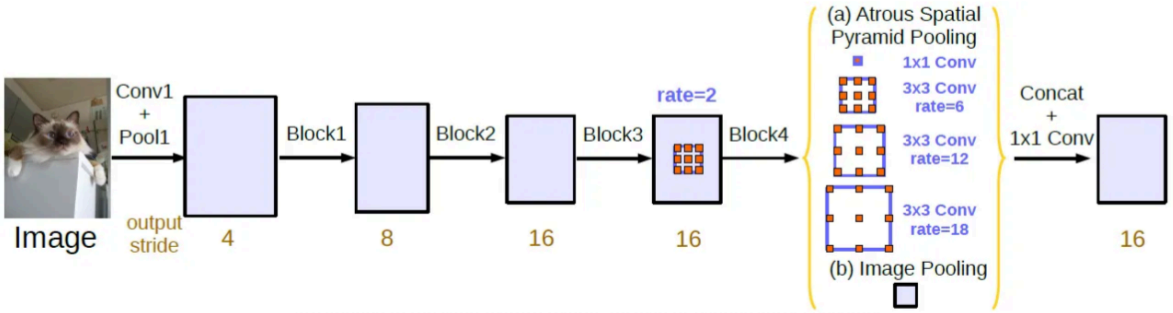
DeepLabV3 utiliza la arquitectura Encoder-Decoder para capturar y procesar características a diferentes escalas y utiliza la convolución atrous o dilatada para capturar información contextual a diferentes escalas.





Atrous Spatial Pyramid Pooling (ASPP)

Utiliza convoluciones atrous a múltiples tasas de dilatación para capturar información contextual a diferentes escalas espaciales. Esto permite que la red tenga un conocimiento más completo y contextual de la imagen, lo que resulta en una segmentación más precisa y detallada.



DeeplabV3

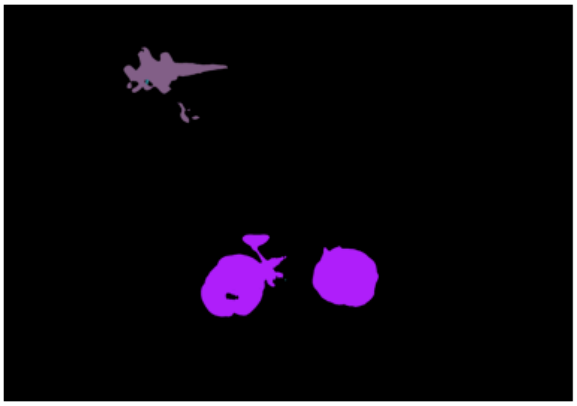
DeeplabV3 es una arquitectura de red neuronal convolucional diseñada para la segmentación semántica de imágenes, entrenado con 21 clases: *background, aeroplane, bicycle, bird, boat, bottle, bus, car, cat, chair, cow, diningtable, dog, horse, motorbike, person, pottedplant, sheep, sofa, train, tvmonitor*. Utiliza una serie de capas convolucionales profundas junto con atrous convolutions (convoluciones dilatadas) para capturar tanto información de contexto global como local en la imagen.

In [8]:
```
deeplabv3_model = torch.hub.load('pytorch/vision:v0.10.0', 'deeplabv3_resnet50',
deeplabv3_model = deeplabv3_model.eval()
```

```
Downloading: "https://github.com/pytorch/vision/zipball/v0.10.0" to /Users/nicol
astorres/.cache/torch/hub/v0.10.0.zip
```

In [9]:
```
deeplab_segmentation(deeplabv3_model, "test/AirplanewithBike.jpg")
```





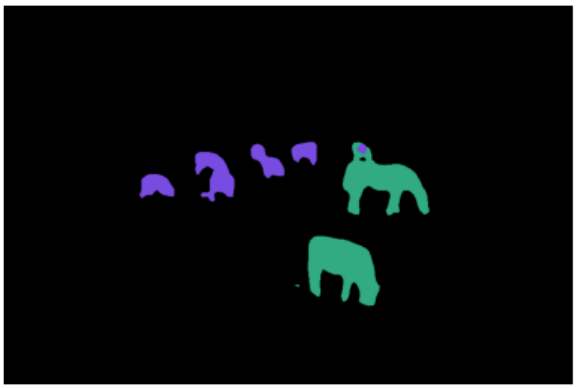
In [10]:
```
deeplab_segmentation(deeplabv3_model, "test/dining.jpg")
```





In [11]:
```
deeplab_segmentation(deeplabv3_model, "test/granja.jpg")
```





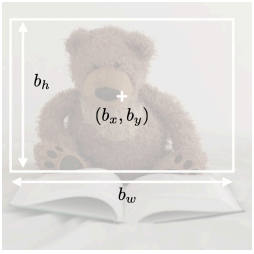
In [12]:
```
deeplab_segmentation(deeplabv3_model, "test/animals.jpg")
```





Object detection

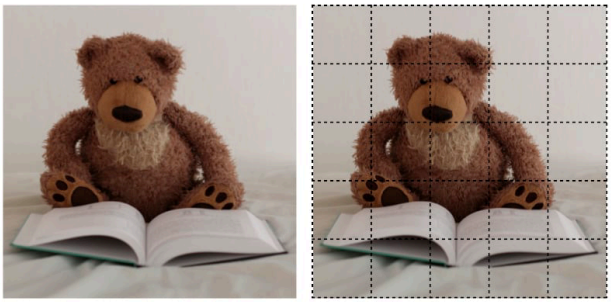
Consiste en identificar y localizar múltiples objetos dentro de una imagen. No solo se clasifican los objetos, sino que también se delimita su ubicación mediante cuadros delimitadores (bounding boxes). Esto permite detectar y etiquetar varios objetos diferentes en una sola imagen.



Faster R-CNN usa dos etapas para detectar objetos, YOLO aborda el problema como una regresión directa y SSD (Single Shot Multibox Detector) combina diferentes escalas de detección.

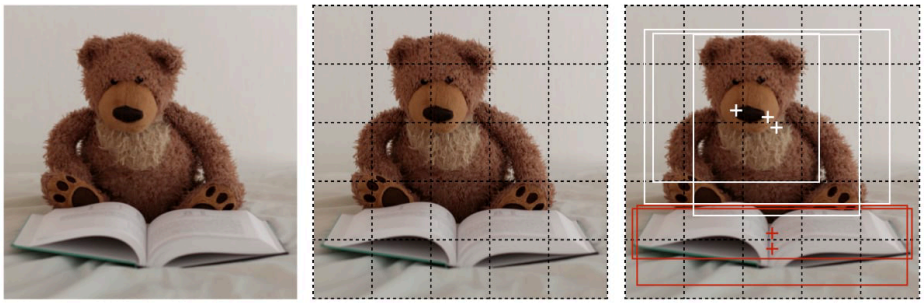
YOLO (You Only Look Once)

YOLO divide la imagen en una cuadrícula, creando una malla que abarca toda la imagen. En lugar de trabajar con regiones propuestas, como otros métodos, YOLO analiza la imagen completa a través de esta cuadrícula.



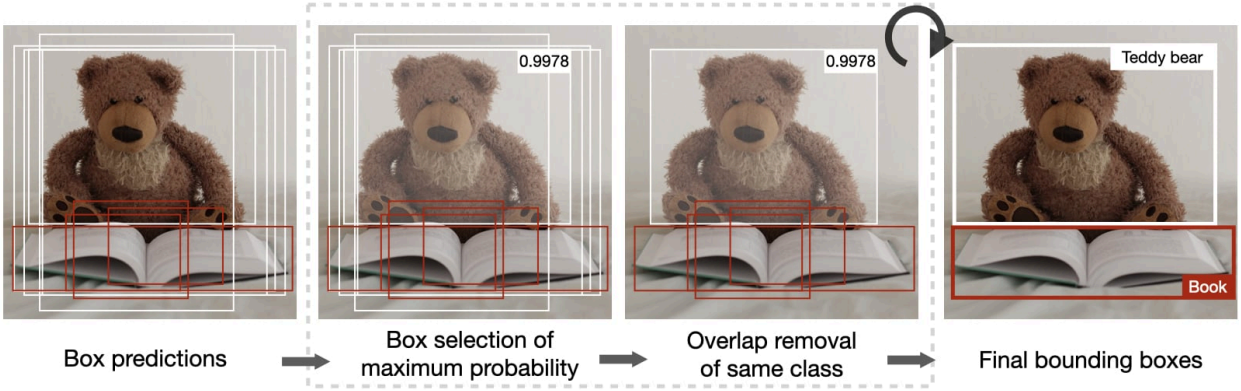
YOLO (You Only Look Once)

Para cada celda de la cuadrícula, YOLO realiza múltiples predicciones. Cada predicción incluye un conjunto de cajas delimitadoras (bounding boxes) y las probabilidades de cada clase que podría contener la caja.



YOLO (You Only Look Once)

Cada caja delimitadora tiene asociada una confianza y también las probabilidades de diferentes clases para ese objeto. Después de las predicciones, YOLO aplica un proceso llamado non-maximum suppression para eliminar las detecciones redundantes. Esto asegura que cada objeto se detecte solo una vez y se filtran las detecciones que se solapan significativamente.



In [13]:
```
yolov5 = torch.hub.load('ultralytics/yolov5', 'yolov5s', pretrained=True, verbos
```

```
requirements: Ultralytics requirements ['gitpython>=3.1.30', 'Pillow>=10.0.1'] n
ot found, attempting AutoUpdate...
Requirement already satisfied: gitpython>=3.1.30 in /opt/anaconda3/lib/python3.
8/site-packages (3.1.40)
Requirement already satisfied: Pillow>=10.0.1 in /opt/anaconda3/lib/python3.8/si
te-packages (10.1.0)
Requirement already satisfied: gitdb<5,>=4.0.1 in /opt/anaconda3/lib/python3.8/s
ite-packages (from gitpython>=3.1.30) (4.0.11)
Requirement already satisfied: smmap<6,>=3.0.1 in /opt/anaconda3/lib/python3.8/s
ite-packages (from gitdb<5,>=4.0.1->gitpython>=3.1.30) (5.0.1)

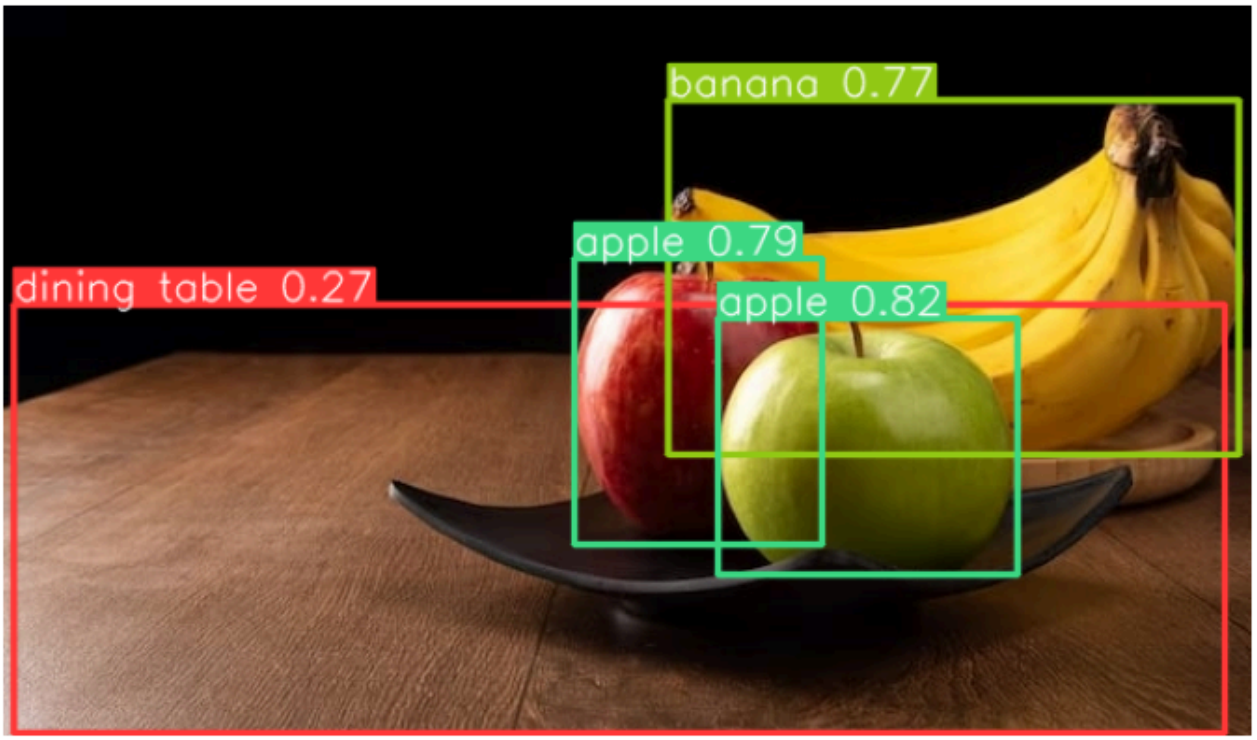
requirements: AutoUpdate success ✅ 7.9s, installed 2 packages: ['gitpython>=3.
1.30', 'Pillow>=10.0.1']
requirements: ⚠️ Restart runtime or rerun command for updates to take effect
```

```
YOLOv5 🚀 2023-11-16 Python-3.10.12 torch-2.1.0 CPU

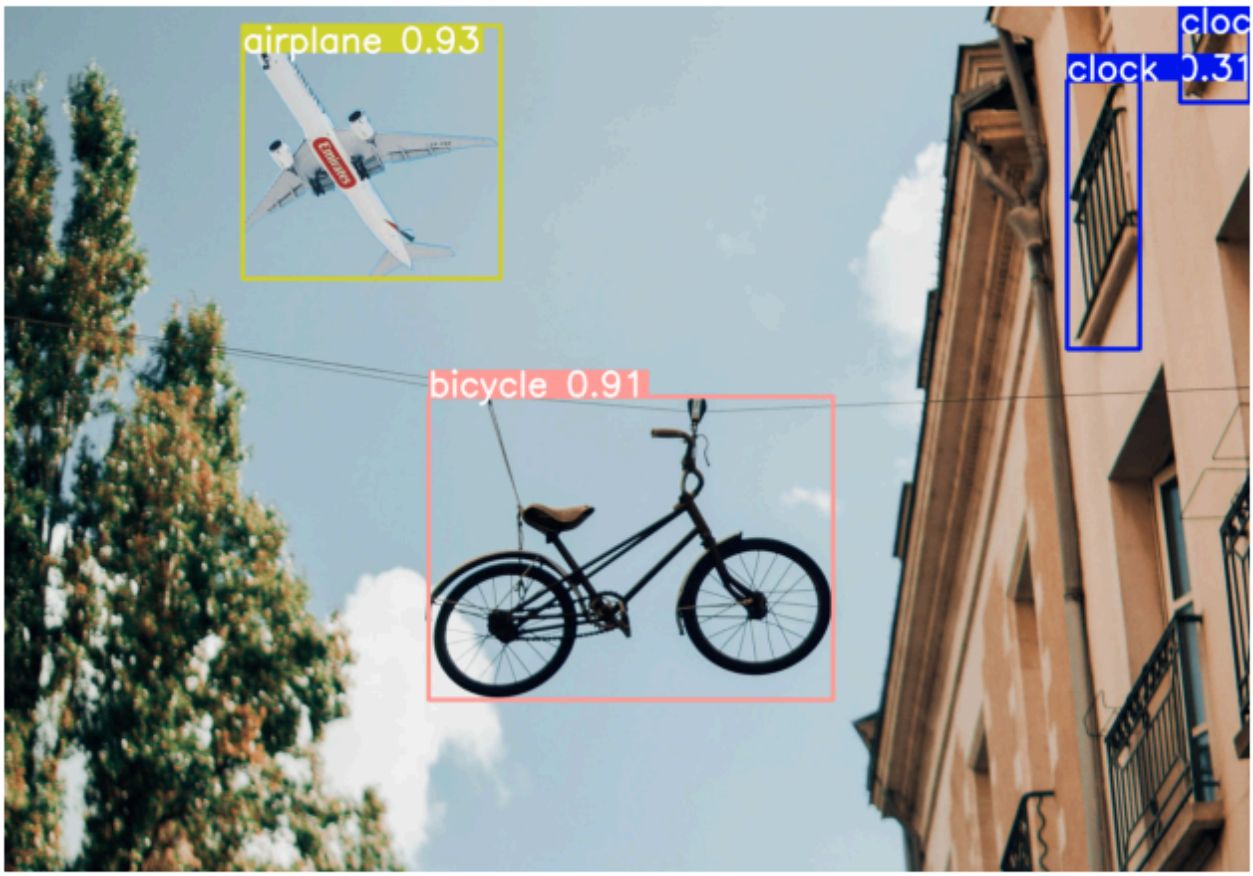
Fusing layers...
YOLOv5s summary: 213 layers, 7225885 parameters, 0 gradients, 16.4 GFLOPs
Adding AutoShape...
```

In [14]:
```
%matplotlib inline
```

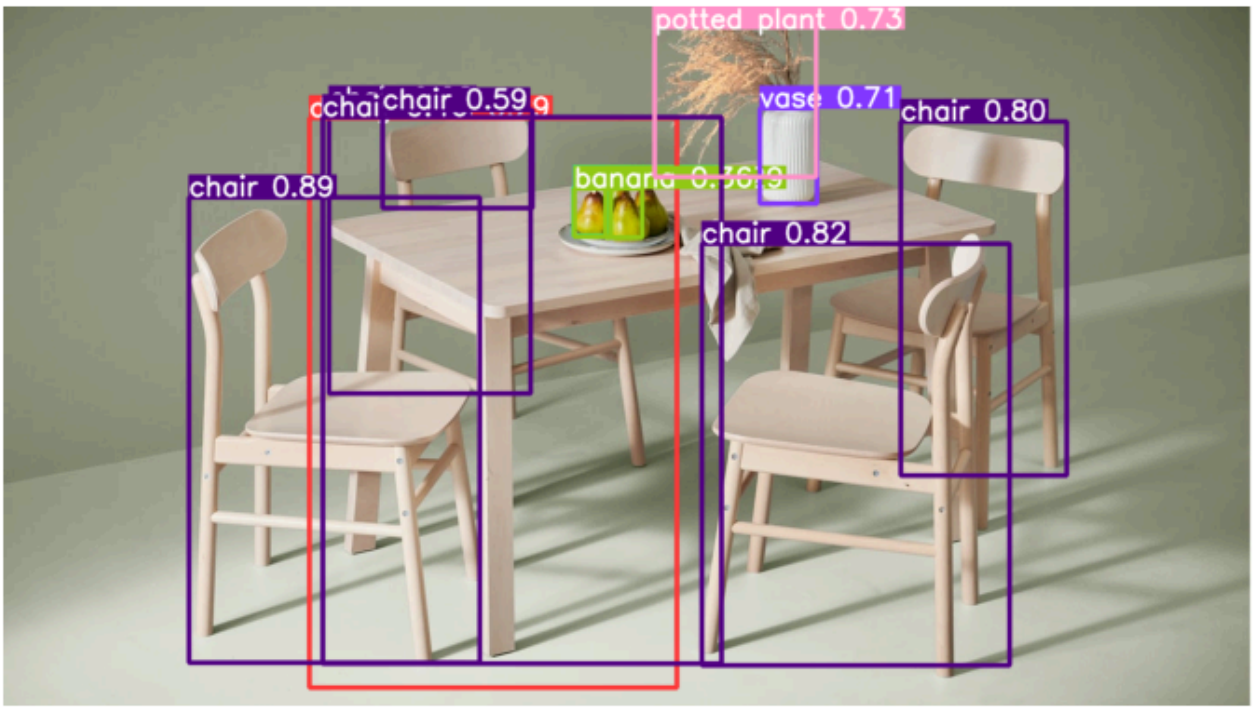
In [15]:
```
plot_result(model=yolov5, path="test/apples-banana.jpg")
```



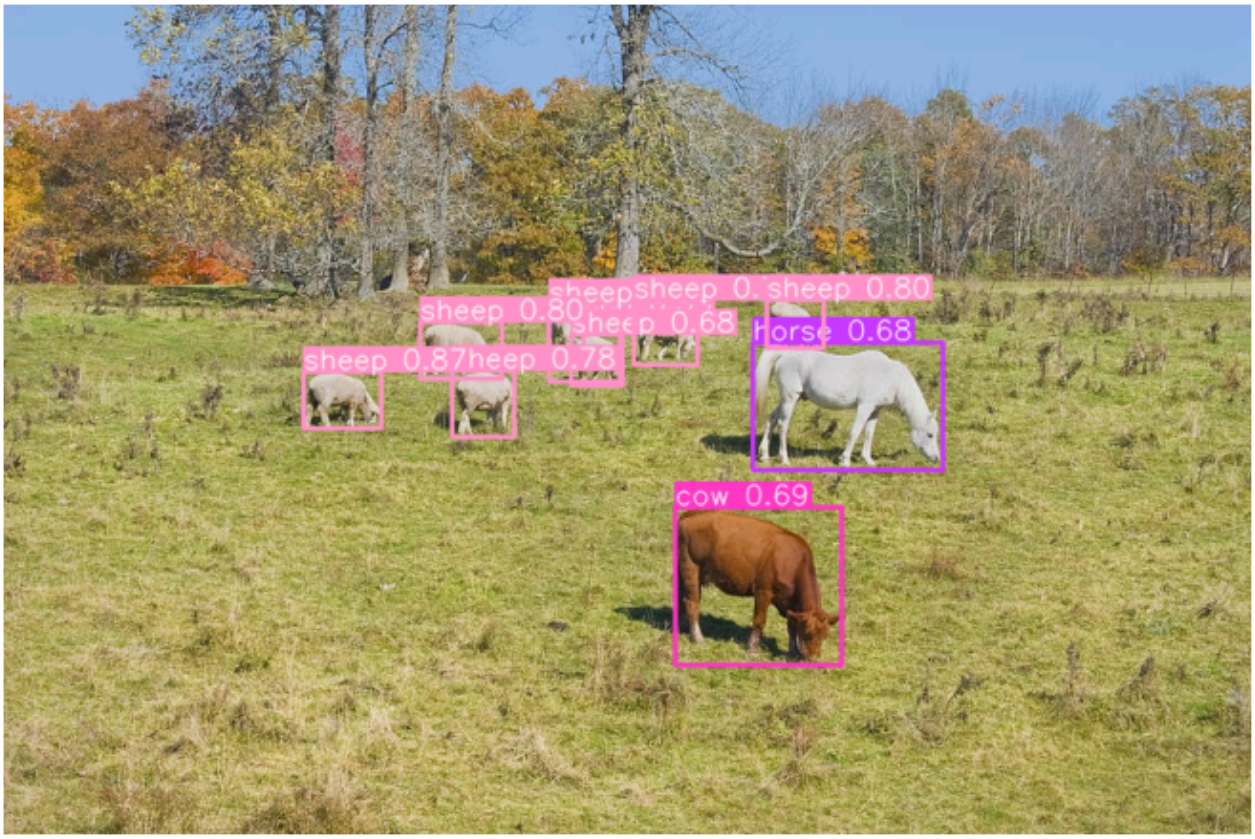
In [16]:
```
plot_result(model=yolov5, path="test/AirplanewithBike.jpg")
```



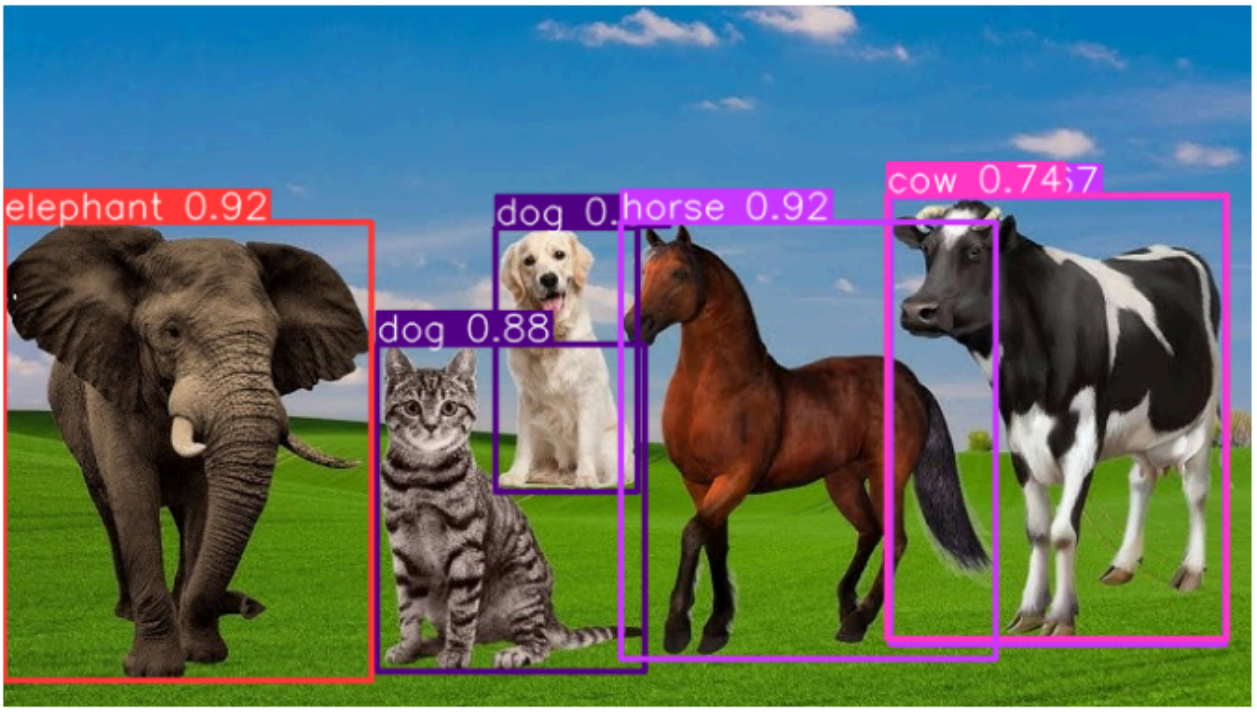
In [17]:
```
plot_result(model=yolov5, path="test/dining.jpg")
```



In [18]:
```
plot_result(model=yolov5, path="test/granja.jpg")
```



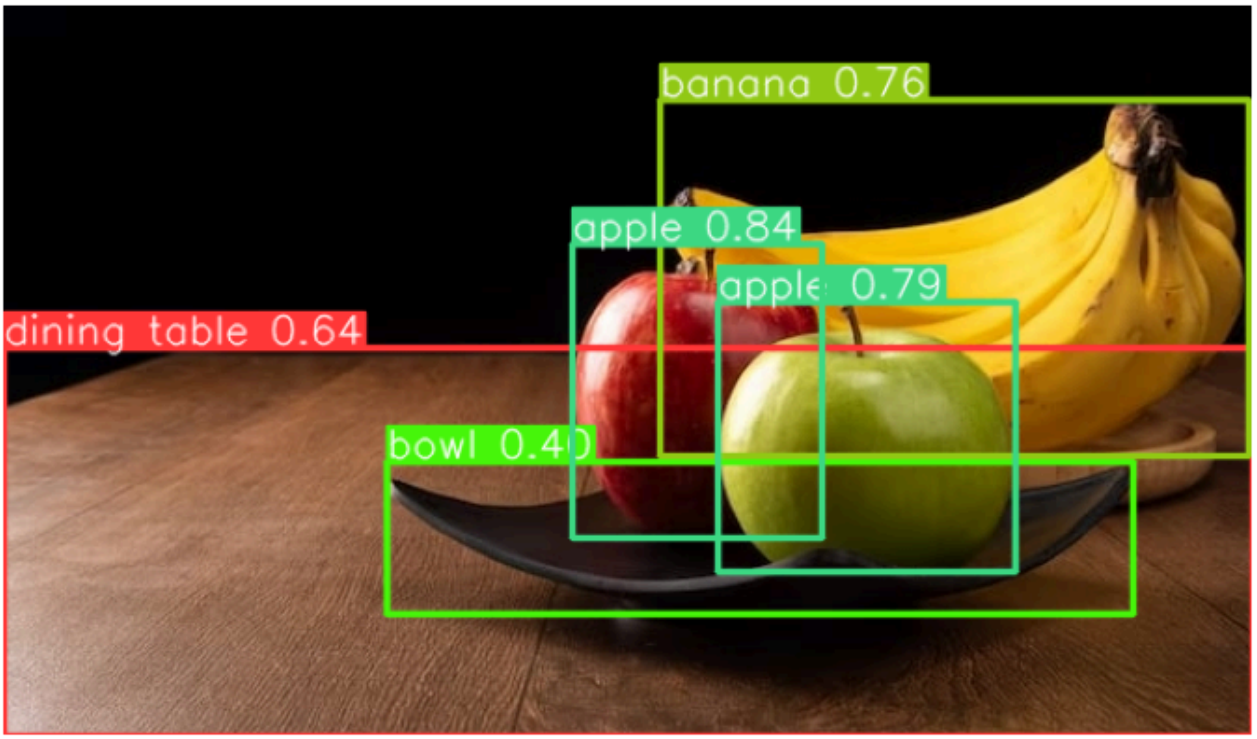
In [19]:
```
plot_result(model=yolov5, path="test/animals.jpg")
```



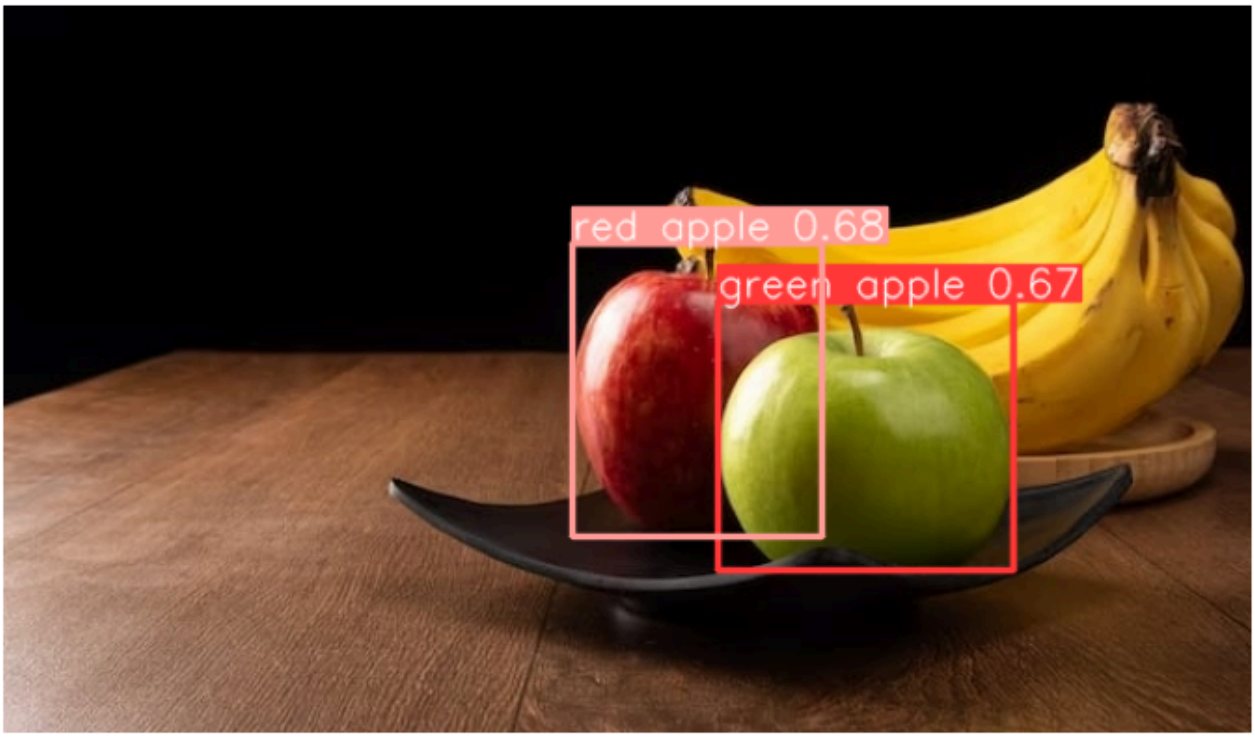
YOLO-World

El YOLO-Modelo Mundial permite detección de vocabulario abierto.

In [20]:
```
plot_yolo_world(path="test/apples-banana.jpg", classes=None)
```

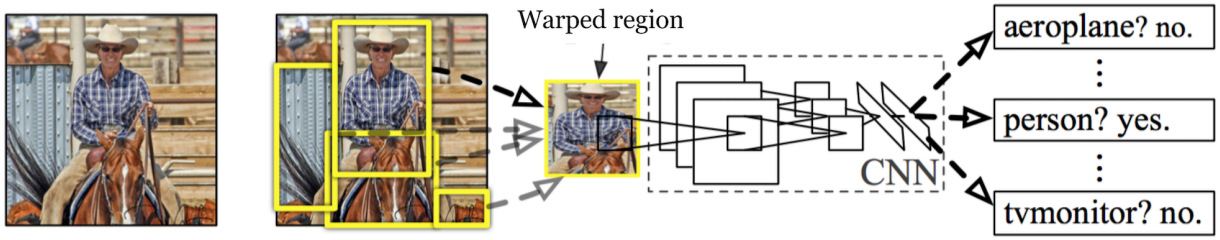


In [21]:
```
plot_yolo_world(path="test/apples-banana.jpg", classes=["green apple", "red appl
```



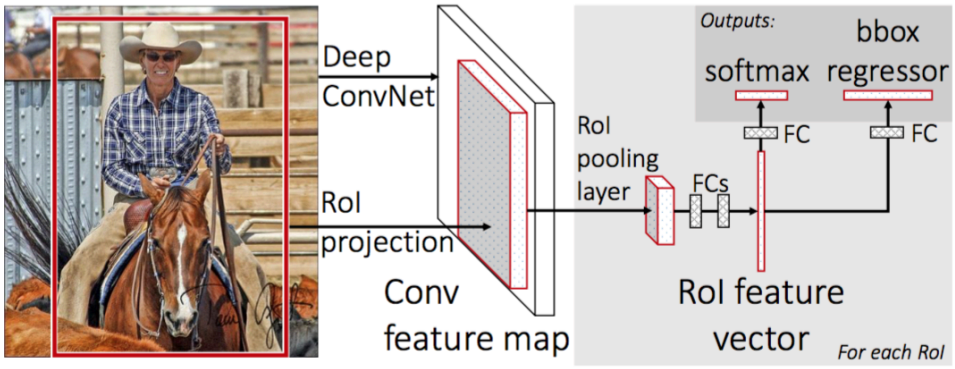
R-CNN

R-CNN (Region-based Convolutional Neural Network) utiliza regiones propuestas para identificar áreas de interés en una imagen. Extrae y clasifica estas regiones mediante una red convolucional, lo que permite detectar y delimitar objetos en la imagen, aunque su proceso es más lento debido al manejo individual de cada región propuesta.



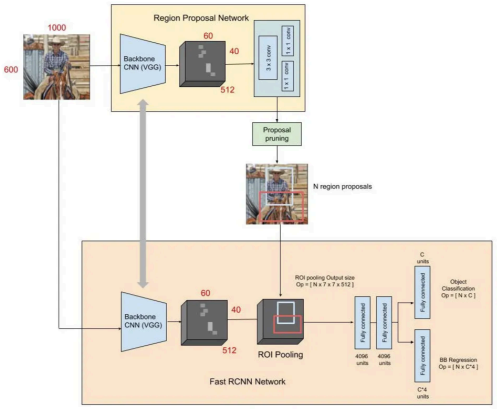
Fast R-CNN

Es una mejora de R-CNN que agiliza el proceso al introducir una red convolucional única para extraer características de la imagen completa en lugar de hacerlo para cada región propuesta individualmente. Luego, utiliza regiones de interés (RoIs) para extraer características específicas y aplicar clasificación y refinamiento, optimizando el tiempo de procesamiento y mejorando la precisión de la detección de objetos.



Faster R-CNN (RPN + Fast R-CNN)

La arquitectura Faster R-CNN consta de RPN como algoritmo de propuesta de región y Fast R-CNN como red de detectores.

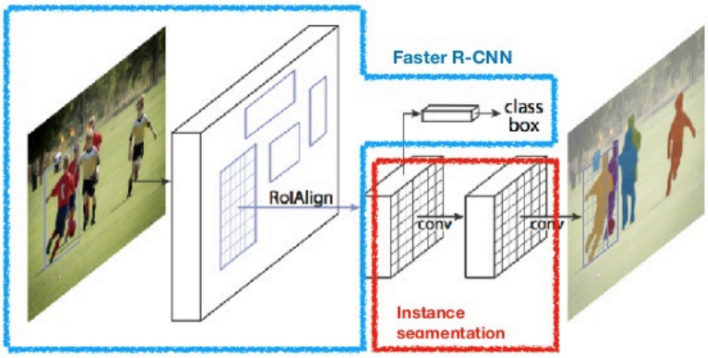


Instance segmentation

Similar a la detección de objetos, pero más avanzada, ya que no solo identifica y delimita la ubicación de los objetos, sino que también segmenta cada instancia individual de un objeto, proporcionando máscaras precisas que representan cada objeto encontrado en la imagen.

Mask R-CNN

Mask R-CNN es una extensión de Faster R-CNN que agrega una rama adicional para realizar segmentación de instancias, proporcionando máscaras de píxeles precisas para cada objeto detectado. Combinando detección de objetos con segmentación a nivel de píxeles, Mask R-CNN permite identificar, localizar y segmentar objetos con gran precisión en imágenes.



MaskRCNN_ResNet50

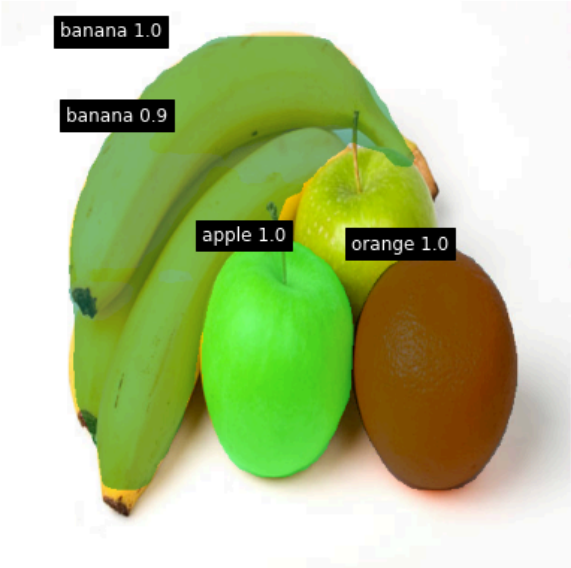
Modelo de redes neuronales convolucionales que combina la detección de objetos con la segmentación de instancias, permitiendo identificar y delimitar los objetos en una imagen, así como generar máscaras de segmentación precisas para cada instancia detectada. Utiliza una arquitectura ResNet50 como extractor de características, lo que le permite alcanzar un equilibrio entre precisión y eficiencia computacional.

In [22]:
```
weights = MaskRCNN_ResNet50_FPN_Weights.DEFAULT
maskrcnn_model = maskrcnn_resnet50_fpn(weights=weights, progress=False)
maskrcnn_model = maskrcnn_model.eval()
```

In [23]:
```
coco_names = ['__background__', 'person', 'bicycle', 'car', 'motorcycle', 'airpl
    'train', 'truck', 'boat', 'traffic light', 'fire hydrant', 'N/A', 'stop sign
    'parking meter', 'bench', 'bird', 'cat', 'dog', 'horse', 'sheep', 'cow',
    'elephant', 'bear', 'zebra', 'giraffe', 'N/A', 'backpack', 'umbrella', 'N/A'
    'handbag', 'tie', 'suitcase', 'frisbee', 'skis', 'snowboard', 'sports ball',
    'kite', 'baseball bat', 'baseball glove', 'skateboard', 'surfboard', 'tennis
    'bottle', 'N/A', 'wine glass', 'cup', 'fork', 'knife', 'spoon', 'bowl',
    'banana', 'apple', 'sandwich', 'orange', 'broccoli', 'carrot', 'hot dog', 'p
    'donut', 'cake', 'chair', 'couch', 'potted plant', 'bed', 'N/A', 'dining tab
    'N/A', 'N/A', 'toilet', 'N/A', 'tv', 'laptop', 'mouse', 'remote', 'keyboard'
    'microwave', 'oven', 'toaster', 'sink', 'refrigerator', 'N/A', 'book',
    'clock', 'vase', 'scissors', 'teddy bear', 'hair drier', 'toothbrush'
]
```

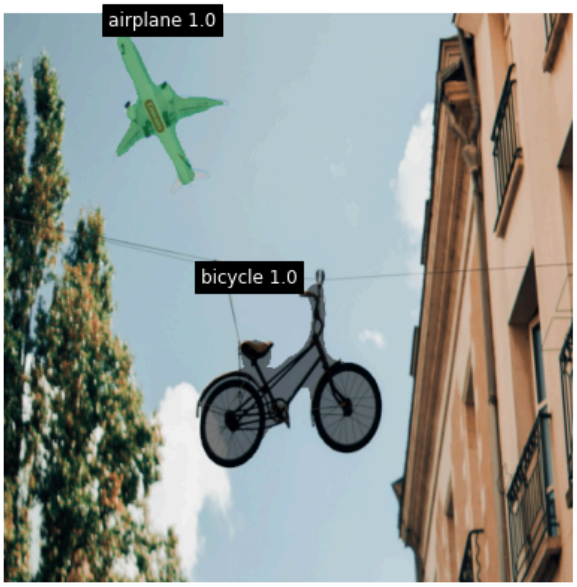
In [24]:
```
mask_segmentation(maskrcnn_model, weights, "test/frutas.jpg", coco_names)
```





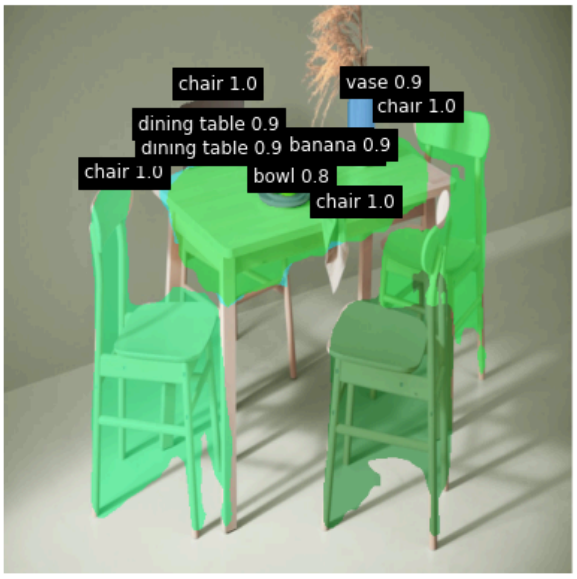
In [25]:
```
mask_segmentation(maskrcnn_model, weights, "test/AirplanewithBike.jpg", coco_nam
```





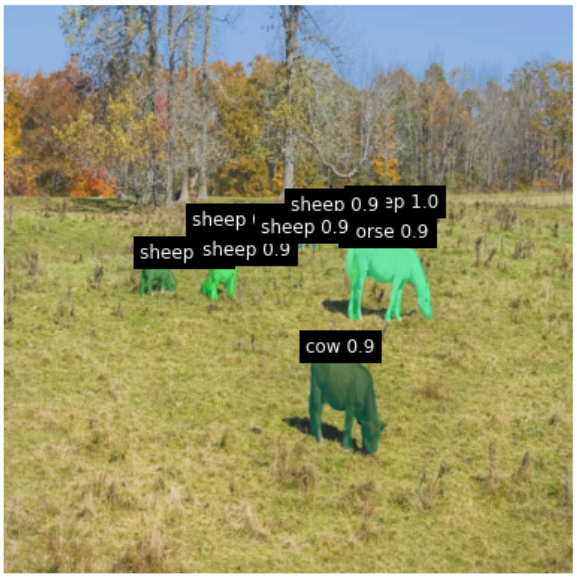
In [26]:
```
mask_segmentation(maskrcnn_model, weights, "test/dining.jpg", coco_names)
```





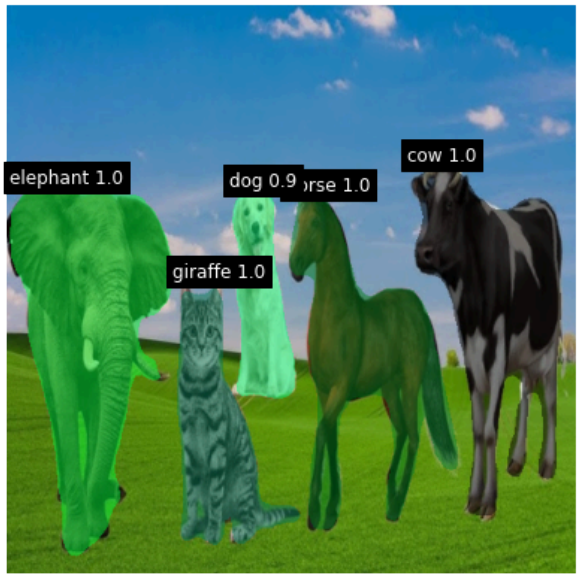
In [27]:
```
mask_segmentation(maskrcnn_model, weights, "test/granja.jpg", coco_names)
```





In [28]:
```
mask_segmentation(maskrcnn_model, weights, "test/animals.jpg", coco_names)
```





Real-Time Detection

In [29]:
```
real_time(video="webcam", task='world', id_cam=0, classes=["hand"])
```

In [ ]:
```

```